

# VARADE: a Variational-based AutoRegressive model for Anomaly Detection on the Edge

Alessio Mascolini<sup>1</sup>, Sebastiano Gaiardelli<sup>2</sup>, Francesco Ponzio<sup>1</sup>, Nicola Dall’Ora<sup>2</sup>, Enrico Macii<sup>1</sup>,

Sara Vinco<sup>1</sup>, Santa Di Cataldo<sup>1</sup>, Franco Fummi<sup>2</sup>

<sup>1</sup>Politecnico di Torino, Turin, Italy, {name.surname}@polito.it

<sup>2</sup>University of Verona, Verona, Italy, {name.surname}@univr.it

## ABSTRACT

Detecting complex anomalies on massive amounts of data is a crucial task in Industry 4.0, best addressed by deep learning. However, available solutions are computationally demanding, requiring cloud architectures prone to latency and bandwidth issues. This work presents VARADE, a novel solution implementing a light autoregressive framework based on variational inference, which is best suited for real-time execution on the edge. The proposed approach was validated on a robotic arm, part of a pilot production line, and compared with several state-of-the-art algorithms, obtaining the best trade-off between anomaly detection accuracy, power consumption and inference frequency on two different edge platforms.

## KEYWORDS

Anomaly detection, Edge computing, Deep learning, Cyber-physical systems, Process monitoring.

## ACM Reference Format:

Alessio Mascolini<sup>1</sup>, Sebastiano Gaiardelli<sup>2</sup>, Francesco Ponzio<sup>1</sup>, Nicola Dall’Ora<sup>2</sup>, Enrico Macii<sup>1</sup>, Sara Vinco<sup>1</sup>, Santa Di Cataldo<sup>1</sup>, Franco Fummi<sup>2</sup>. 2024. VARADE: a Variational-based AutoRegressive model for Anomaly Detection on the Edge. In *61st ACM/IEEE Design Automation Conference (DAC ’24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3655691>

## ACKNOWLEDGMENTS

The work has been partially supported by PRIN 2022T7YSHJ SMART-IC - Next Generation EU project. This manuscript reflects only the Authors’ views and opinions, neither the European Union nor the European Commission can be considered responsible for them.

## 1 INTRODUCTION

In any production context, the downtime of a machine due to the sudden breakdown of mechanical, hydraulic, or electrical components leads to severe losses in terms of time and money. For this reason, efforts are spent for the early detection of any irregular behavior of the production line to avoid sudden stops, enable specific preventive maintenance actions, and reduce the environmental impact. This evolution is enabled by transforming traditional production machinery into Cyber-Physical Systems (CPSs), where

sensor devices, communication technologies, and data analytics cooperate to manage production failures in advance [25].

In an industrial CPS scenario, the most crucial resource is the availability of data reflecting the different aspects of production. Such data consist of multiple interdependent variables rapidly evolving over time, thus falling under the typical definition of *Multivariate Time Series (MTS)* [14]. After collection, the time series, originated by heterogeneous sensors and data sources, are integrated through Industrial Internet of Things (IIoT) technologies and made available for anomaly detection, visualization, and analysis [27].

Although extensive research has been carried out on Multivariate Time Series Anomaly Detection (MTSAD), current solutions typically lack the flexibility and scalability that is required for an effective real-time deployment [19, 28]. In most proposed solutions, the raw data are in fact streamed through the IIoT network to a cloud platform [11], where an expert data-driven system is in charge of the anomaly detection stage. This typically results in high latency owed to communication overhead [2]. Unlike Internet of Things (IoT) networks, IIoT networks are characterized by sensors transmitting a massive amount of data that must be processed in real-time [8, 20]. The order of magnitude of the transmitted data can be GB/s for large production plants, making cloud processing impracticable due to the bandwidth requirements and impairments [20]. All such considerations highlight that anomalies should be detected *as soon as possible* and *as close as possible* to the monitored CPS, preferably with real-time or near real-time response, rather than on the cloud. This makes *edge computing* strategic to maintain the overall system functionally safe [14, 26].

On top of these considerations, in our study, we propose VARADE, a novel real-time and *edge-friendly* anomaly detection solution that provides a new efficient training paradigm for *light* MTSAD. The autoregressive framework of VARADE allows handling streaming data with minimal latency. Furthermore, its variational formulation obtains the best compromise between model compactness and anomaly detection accuracy, making it best suited for real-time execution on the edge.

To prove the effectiveness of VARADE in a real industrial CPS scenario, we employ a collaborative robot working in a fully-fledged manufacturing line and providing a continuous stream of heterogeneous sensor data. On this testbed, we compare VARADE with a comprehensive set of state-of-the-art *light* (i.e., edge-suitable) MTSAD solutions on two different edge platforms. Our experiments demonstrate that VARADE performs well even with limited computational resources, with an optimal balance between required power, anomaly detection accuracy, and inference frequency of the model, which can be varied according to the industrial machinery being monitored.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC ’24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06.

<https://doi.org/10.1145/3649329.3655691>

This paper is organized as follows. Section 2 presents the necessary background and related works. Section 3 details the proposed VARADE anomaly detection method, as well as the other benchmarked solutions. Section 4 describes the collaborative robot case-study and discusses the experimental results. Finally, section 5 draws our concluding remarks.

## 2 BACKGROUND AND RELATED WORKS

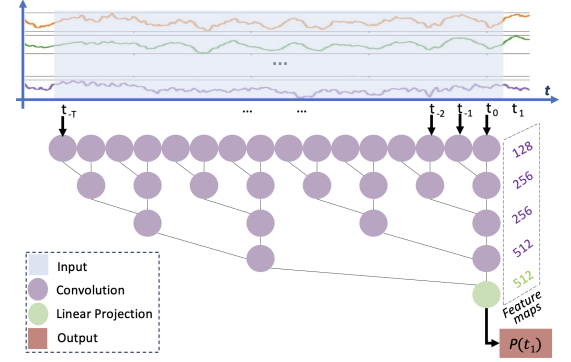
MTSAD scenarios are best addressed with Deep Learning (DL) methodologies, that recently proved to be more effective in tackling complex anomalies in MTS data [13, 23] than traditional anomaly detection methods (e.g., based on clustering or statistical indexes [29]). Nonetheless, most DL-based solutions present major drawbacks in terms of required data transmission and/or high computational cost [16, 28]. On the other hand, *light* MTSAD models compatible with edge computing are typically based on tiny and scaled Convolutional Neural Networks (CNNs), that need to be trained with huge sets of annotated anomalies [16, 19, 24, 28]. Collecting and annotating such training sets is however unfeasible in most industrial applications [13, 19].

To circumvent this problem, the most promising approach is to learn the characteristics of a “normal” behavior from a large amount of non-anomalous data so as to be able to identify any events that significantly deviate from the normality, with three different strategies: i) forecasting-based, ii) reconstruction-based, and iii) outlier detection methods.

*Forecasting-based* methods learn to predict a number of time steps leveraging a current context window. Then, they compare the predicted values with the observed ones to identify anomalies [23]. A large number of studies in this group employ autoregressive Long Short-Term Memory (LSTM) networks, a type of recurrent network able to learn long-term time dependencies in multivariate data [4, 12, 17]. A recent work leverages instead a forest of gradient boosted regression trees to detect anomalies in a Digital Twin-driven industrial context, by examining the residuals from the forecasts of an ensemble of weak predictors [9].

*Reconstruction-based* methods encode the characteristics of a normal time series into a latent representation and learn to reconstruct new data starting from it. The reconstruction error is then exploited to discriminate the anomalous values from the normal ones. The most popular methods in this group are built on top of autoencoders (AEs), encoder-decoder neural networks where the encoder learns a compressed version of the input data, and the decoder learns to recreate the input starting from the encoded representation. Among the others, [10] employed convolutional AEs for anomaly detection in an IoT-inspired environment, and proved that reducing the size, complexity, and training cost of the AE did not lower its ability to identify anomalies.

*Outlier detectors* identify anomalies based on their dissimilarity from regular data points in the feature space. Popular edge-friendly examples in this group are based on k-Nearest Neighbors (kNN), identifying anomalous values based on the distance from their neighbours, and Isolation Forest, that uses the number of binary splits necessary for an ensemble of decision trees to isolate the point from the rest of the data [15].



**Figure 1: Architecture of VARADE.** Current ( $t_0$ ) and past time steps ( $t_{-1} \dots t_{-T}$ ) are processed by a cascade of convolutional layers and a final linear projection. The output is the estimated probability distribution of the next time step,  $P(t_1)$ .

## 3 METHODS

### 3.1 Proposed solution

VARADE works to strike a balance between traditional techniques, that offer quick inference but with limited accuracy, and DL models, that learn complex patterns but require substantial computational resources, not available at the edge. Our design choice is to employ a *forecasting-based autoregressive framework*: by predicting samples one at a time based on previous ones, this framework is naturally suited to handle streaming data with minimal latency.

Figure 1 illustrates in principle the proposed architecture. The model takes as input the samples at the current ( $t_0$ ) and past time steps ( $t_{-1}, \dots, t_{-T}$ , with  $T = 16$  for visualization purposes), and passes them through a set of convolutional layers with ReLU activations and a linear projection, to finally predict a single future time step,  $t_1$ . The reason behind this architectural choice lies in the consideration that model inference speed of CNNs is commonly limited by memory bandwidth, especially with SIMD implementations for CPUs and CUDA kernels for GPUs [5]. By using convolutions with kernel size and stride of 2, we obtain that the time-dimension is halved at every new layer, leading to very limited memory usage and bandwidth requirements compared to the number of parameters, and hence to faster inference. On the other hand, the number of feature maps is doubled every two layers (see Figure 1), helping the network to learn more complex and abstract features.

Conventional forecasting-based anomaly detectors work by considering an *anomaly score*, measured as the euclidean norm between the forecasted value and the measured one. In our experiments, we have observed that a DL-based autoregressive model compact enough for real-time execution on edge fails to deliver satisfactory forecasting performance, dramatically affecting the quality of the anomaly scores. This lead us to a probabilistic approach, where the model outputs a probability distribution of the possible values for the next data point in the sequence ( $P(t_1)$  in Figure 1).

Predicting a probability distribution using a neural network is a complex problem, which can be greatly simplified by constraining the distribution to be Gaussian. This approach, known as *variational inference* [3], leads to a simpler optimization problem where the objective is to find the mean and variance which minimize the

loss function (details will follow). The additional advantage of the Gaussian constraint is that the variance can be interpreted as the uncertainty of the prediction: since we expect the model to be more confident in its prediction when the system is operating normally, and less confident when an anomaly is occurring, the variance can be directly used as an *anomaly score*.

Summarizing, the proposed architecture is composed by  $N$  convolutional layers, with time-dimension halved at every layer. Hence,  $N$  strictly depends on the input window size  $T$ . In our work, we set  $T = 512$ , resulting into a total of 8 layers. Conversely, the number of feature maps is doubled every two layers starting from 128, which leads to 1,024 in the final layer. The output mean and variance values of the estimated probability distribution  $P(t_1)$  are obtained at last by linear projection.

### 3.2 Derivation of the loss function

As loss function we employ the inverse of the Evidence Lower Bound (ELBO), that provides a lower bound on the log evidence,  $\log p(x)$ , where  $x$  represents the observed data. Thus, maximizing the ELBO leads to a better approximation of the true posterior.

The ELBO can be decomposed into two terms:

- The expectation of the log-likelihood under the approximate posterior, which pushes the approximate distribution to put more probability mass on configurations of the latent variables that explain the observed data well.
- The negative divergence between the approximate and prior distribution, which encourages the approximate distribution to be close to the prior.

By presuming a Gaussian distribution, our model predicts both the mean and the logarithm of the distribution's variance. We opt for the logarithm over the simple variance, as the latter can only be positive. Hence, the reconstruction loss is essentially computing the negative log-likelihood of the observed data under a Gaussian distribution assumption.

Let us assume that our data  $y$  is normally distributed with a mean of  $\mu$  and a variance of  $\sigma^2$ . The probability density function (PDF) of a normal distribution is given by:

$$p(y|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right) \quad (1)$$

Taking the negative logarithm of the PDF to get the negative log-likelihood (NLL), we have:

$$NLL(y|\mu, \sigma^2) = -\log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right)\right) \quad (2)$$

After simplifying the above equation, we get:

$$NLL(y|\mu, \sigma^2) = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y-\mu)^2}{2\sigma^2} \quad (3)$$

Given that  $\log(2\pi)$  is just a constant, we can ignore it during optimization (as it depends on the derivative of the loss, and the derivative of a constant is zero). This simplifies to:

$$NLL(y|\mu, \sigma^2) = \frac{1}{2} \log(\sigma^2) + \frac{(y-\mu)^2}{2\sigma^2} \quad (4)$$

So, in our case, the reconstruction loss we use is:

$$L_{\text{recon}} = \frac{1}{2} \left( \log(\sigma_{\text{pred},i}^2) + \frac{(y_i - \mu_{\text{pred},i})^2}{\sigma_{\text{pred},i}^2} \right) \quad (5)$$

where  $\sigma_{\text{pred}}^2$  represents the predicted variance,  $\mu_{\text{pred}}$  the predicted mean and  $i$  the current step. This formula encourages our model to predict a distribution close to the actual data.

The next part of the loss calculation introduces the Kullback–Leibler (KL) divergence, which quantifies the difference between our predicted distribution and our prior, a standard Gaussian distribution. This is computed as:

$$D_{KL} = -\frac{1}{2} \left( 1 + \log(\sigma_{\text{pred}}^2) - \mu_{\text{pred}}^2 - \sigma_{\text{pred}}^2 \right) \quad (6)$$

The  $D_{KL}$  term encourages the model to predict the data mean and variance when it is uncertain. This helps regularize our model and is critical to employ our anomaly detection method.

The final loss function  $L$  is a weighted sum of the reconstruction loss  $L_{\text{recon}}$  and the KL divergence  $D_{KL}$ :

$$L = L_{\text{recon}} + \lambda D_{KL} \quad (7)$$

Thanks to the KL divergence term, our model learns to predict a higher variance when it is uncertain about the next value, and a low variance when it is confident. During inference, the mean prediction is removed, and the variance is directly used as an *anomaly score*: the higher the score, the larger the detected anomaly.

### 3.3 Baseline solutions

As a baseline for our proposed method, in this study, we implement and analyze a representative sample of *light* anomaly detectors that have been successfully deployed in edge computing scenarios, by considering the approaches in section 2.

- *Autoregressive Long Short-Term Memory (AR-LSTM)*. A recurrent architecture featuring 5 LSTM recurrent layers with 256 feature maps each, followed by 2 fully connected layers. The anomaly score is then calculated as the euclidean norm of the difference between predicted and real value, as in many previous works [4, 12, 17, 21]. To find the best configuration for our specific task, we follow the memory-efficient paradigm introduced by [22], and pick a number of layers equal to 5 based on past experiments at a similar window size [1].
- *Gradient Boosted Regression Forest (GBRF)*. The technique presented in [9], with minor modifications to boost the anomaly detection capabilities: the number of decision trees is increased from 5 to 30 and the dimensionality reduction step is removed. The anomaly score is computed in the same way as for AR-LSTM.
- *Autoencoder (AE)*. A convolutional autoencoder featuring 6 ResNet blocks [7]. The anomaly score is the euclidean norm of the difference of reconstructed and real value.
- *kNN*. Past works show kNN as the best performing nearest neighbour based algorithm for anomaly detection, with anomaly score computed either as the average or the maximum distance from the neighbors [6]. We employ maximum distance with  $k=5$ , as it has the best compromise between accuracy and execution time.
- *Isolation Forest*. An ensemble of 100 individual decision trees, that isolate each data point into a leaf. The anomaly score of a data point is based on the average path length [15]. As recommended by [15], we use a contamination value of 0.1, which defines the proportion of outliers in the dataset.

### 3.4 Implementation details

All the models were implemented in TensorFlow 2.11.0 and Sklearn 1.1.2. For a fair comparison, all the anomaly detection frameworks were trained in the same experimental conditions and implementing hyperparameters tuning strategies<sup>1</sup>. More specifically: the neural network-based frameworks were optimized using Adam with a fixed  $10^{-5}$  learning rate. GBRF and Isolation Forest were trained using the mean squared error criterion and recursive binary splitting, by strictly following the respective reference papers.

## 4 INDUSTRIAL CASE STUDY

### 4.1 Kuka anthropomorphic manipulator

To create a realistic scenario for the anomaly detection methods, we focused our case study on a KUKA LBR iiwa collaborative industrial robot, part of a fully-fledged production line<sup>2</sup>. The robot performs pick and place operations and it is controlled by a Simatic S7-1200 Programmable Logic Controller (PLC), directly connected to the robot through a hard-wired field bus. The PLC runs an OPC Unified Architecture (OPC UA) server, that exposes the KUKA state and functionality as services: the activation of such services in a given order constitutes a production process.

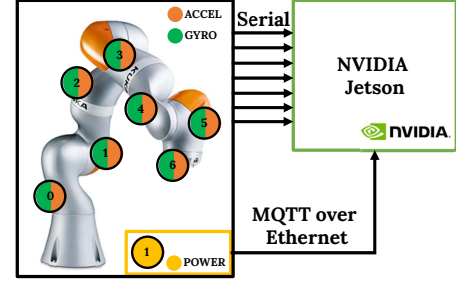
The KUKA robot allows collecting the robot’s parameters through its programming interface. However, this limits the frequency with which such parameters can be collected to 5 Hz. At higher frequencies, queries interfere with the controlling process, causing stuttering in the robot trajectories. For this reason, we instrumented the KUKA robot with seven Inertial Measurement Unit (IMU) sensors (DFRobot SEN0386), one on each robot joint, to measure the joint’s angle, acceleration, and angular velocity. These sensors send data at 200 Hz on a serial wire after applying a Kalman filter to reduce noise. In addition to physical data, we collected also extra-functional data from a single-phase energy meter (Eastron SDM230) monitoring the energy consumption of both the robot and the industrial PC. This energy meter is connected through a hard-wired Modbus with an industrial ESP-32 (Olimex ESP32-EVB), collecting and sending data to a MQTT broker via Ethernet.

Figure 2 depicts the experimental setup, consisting of: the KUKA robot, seven IMU sensors, an energy meter, and an embedded board connected to the sensors and executing the anomaly detection model (further described in Section 4.4).

### 4.2 Data stream characterization

The data stream collected from the robotic manipulator consist of 86 channels in total (reported in Table 1), including signals to monitor the action currently performed by the robot (*i.e.*, *action ID*), its kinematic behavior (*Joint Channels*) and its extra-functional parameters (*Power Channels*) [18].

The Joint Channels consist of data related to the seven joints collected from the IMUs sensors, each having the same eleven components that monitor various aspects of motion and temperature. Originally, the IMU collect angles in the  $[-180, +180]$  °C range, causing high value changes when rotating near the two extremes. Since this may be a source of confusion for pattern recognition



**Figure 2: Case study setup.** The KUKA manipulator is instrumented with different sensors: 7 accelerometers with six axes (one for each joint) and one single-phase power meter. The sensors are connected directly with an embedded board for detecting different classes of anomalies.

techniques, we had the orientations converted to quaternions, a 4-dimensional coordinate system commonly used in robotics.

The Power channels consist of eight quantities monitored by the energy meter. These channels allow detecting anomalies that could be transparent with respect to the robot trajectories, such as high power draw from a motor.

### 4.3 Experimental setup

To train VARADE and the baseline anomaly detection models, we created a dataset by recording the robot performing 30 unique actions (*i.e.*, its machine services) executed in a cycle for a total duration of 390 minutes. The resulting dataset contains all the possible actions supported by the robot, distributed uniformly within its duration. This allows the offline training of the anomaly detection models on the “*normal behavior*” of the robot in all the possible production processes supported by the manufacturing system. Given the diverse nature of the anomaly detection models, the collected

**Table 1: Channels description:** for each sensor considered, the related variables are listed. The <X> in the variable name is a label representing the [0,6] index of the joint in which the corresponding IMU sensor is placed on the robot.

|                | Channel name      | Unit             | Description                |
|----------------|-------------------|------------------|----------------------------|
| Joint Channels | action ID         | -                | Robot action ID            |
|                | sensor_id_X_AccX  | m/s <sup>2</sup> | X-axis acceleration        |
|                | sensor_id_X_AccY  | m/s <sup>2</sup> | Y-axis acceleration        |
|                | sensor_id_X_AccZ  | m/s <sup>2</sup> | Z-axis acceleration        |
|                | sensor_id_X_GyroX | deg/s            | X-axis angular velocity    |
|                | sensor_id_X_GyroY | deg/s            | Y-axis angular velocity    |
|                | sensor_id_X_GyroZ | deg/s            | Z-axis angular velocity    |
|                | sensor_id_X_q1    | -                | Quaternion orient. comp. 1 |
|                | sensor_id_X_q2    | -                | Quaternion orient. comp. 2 |
|                | sensor_id_X_q3    | -                | Quaternion orient. comp. 3 |
|                | sensor_id_X_q4    | -                | Quaternion orient. comp. 4 |
|                | sensor_id_X_temp  | °C               | Temperature                |
| Power Channels | current           | A                | Current                    |
|                | frequency         | Hz               | Frequency                  |
|                | phase_angle       | degree           | Phase angle                |
|                | power             | W                | Power                      |
|                | power_factor      | -                | Power factor               |
|                | reactive_power    | VAr              | Reactive power             |
|                | voltage           | V                | Voltage                    |

<sup>1</sup>Source code and dataset available at <https://gitlab.com/AlessioMascolini/varade>

<sup>2</sup>Industrial Computer Engineering Laboratory - <https://www.icelab.di.univr.it/>

data are normalized in the range  $[-1, 1]$  based on the minimum and maximum values of each sensor's data, ensuring that all the features have equal importance avoiding unfair comparison.

To test the trained models in real-time conditions, we designed a "collision experiment" of 82 minutes in total. During this experiment, the robot performed all the 30 possible actions. During the robot operations, 125 collision anomalies were randomly generated by a human operator, by manually interfering with the robot during its movement in a very limited timeframe. This simulates sudden collisions between a human worker (or an object) and the robot, which is a realistic hazardous situation in a production line.

To test the suitability to an edge scenario, we selected two edge devices, connected to the robotic system depicted in Figure 2: a Nvidia Jetson Xavier NX (with 6 cores and 16 GB of RAM) and a Jetson AGX Orin (with 12 cores and 32 GB of RAM). Each anomaly detection model has been tested by a software script that continuously reads data from the sensors, prepares the data by applying a preprocessing function, and calls the inference function.

During each test, the anomaly detection accuracy was evaluated in terms of Area Under the Receiver Operating Characteristic Curve (AUC-ROC) value. The ratio is to interpret an anomaly detector as a binary classifier, where points are classified as anomalous if the anomaly score exceeds a certain threshold. The Receiver Operating Characteristic (ROC) curve plots the true positive rate against the false positive rate at varying values of this threshold, and the area under this curve provides a single threshold-less  $[0, 1]$  measure of the algorithm's ability to identify the anomalous data points.

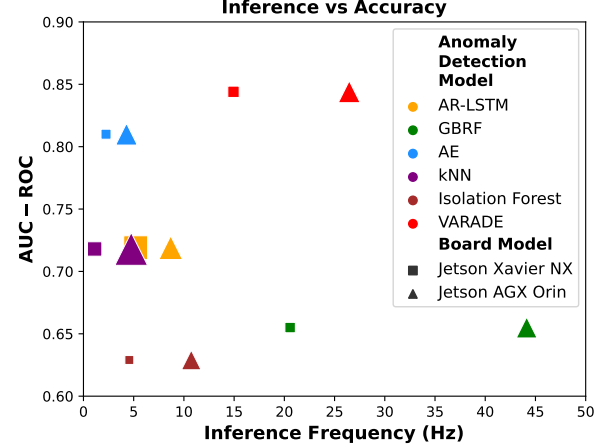
Besides the AUC-ROC score, we measured the inference frequency, and we collected all the most relevant board's metrics (e.g., power consumption, RAM usage, GPU RAM usage) by exploiting the *jetson-stats* library. These metrics were collected not only during the execution of the anomaly detection tasks, but also with the boards in *Idle state* for 6 minutes: the mean value was computed as baseline to evaluate the load introduced by the anomaly detection models *w.r.t.* the standard state.

#### 4.4 Experimental results

Table 2 reports the results obtained by VARADE and by the baseline detectors described in Section 3.3 on the two selected edge devices.

**Jetson Xavier NX.** VARADE placed first in terms of accuracy, with an AUC-ROC score of 0.84, with an improvement of 4% *w.r.t.* the second most performing model (AE) and of 13% *w.r.t.* the third most performing one (AR-LSTM). Interestingly, these improvements correspond also to a higher inference frequency, improved 7 times *w.r.t.* AE and 3 times *w.r.t.* AR-LSTM.

Considering inference frequency, VARADE placed second, with 15 Hz against 20 Hz obtained by GBRF. However, when considering the AUC-ROC scores, VARADE offers an improvement of almost 20% *w.r.t.* GBRF. Looking at RAM usage, we note that all the models use almost the same amount of memory, while VARADE uses a higher amount of GPU RAM (500 MB). This is not a limitation for the applicability of VARADE, as the total amount of memory used is under 40%, thus leaving enough space for larger anomaly models or other applications. Another important parameter for an edge device is the power required to operate, as the device could operate in conditions with limited power. Almost all the models



**Figure 3: Inference Frequency vs Accuracy of the anomaly detection models (identified by the marker color). Marker shape represents the adopted edge device (square for Jetson Xavier NX, and triangle for Jetson AGX Orin), while marker size is proportional to power consumption.**

have comparable performance in terms of power consumption, except AR-LSTM (for its high usage of the GPU), and kNN (for its high usage of CPU).

**Jetson AGX Orin.** Analyzing the results obtained on the Jetson AGX Orin, we can note that the results are similar to the ones obtained with the Jetson Xavier NX but with a different scale. We can see that inference frequency is more or less doubled for all the anomaly detection models, but the overall ranking remains the same, with GBRF in the first position and VARADE in the second position. A significant difference relies in the GPU usage, as in this case the TensorFlow planner decided to run kNN and Isolation Forest on the CPU due to the higher number of CPU cores.

Figure 3 highlights the characteristics of the different configurations by depicting the ratio between the Inference Frequency and the AUC-ROC scores of the tested anomaly detection models.

Looking at the results obtained on the two boards, we can draw the following conclusions. The two anomaly detection models less suitable (in our case study) for anomaly detection on the edge are the kNN and the AR-LSTM. kNN is an algorithm that cannot fully benefit from GPU parallelism (especially with a few channels, as in our case study). On one side, this problem can be solved by exploiting CPU parallelism, but on the other side, edge devices have limited computation power, leading to high power draw and limited CPU available to run other jobs. AR-LSTM is based on a memory-intensive architecture that is not designed to work in a constrained environment with high throughput requirements. In fact, with both the boards, we can note a high GPU usage, which could seem a positive factor but leads in fact to low inference speed.

At the same time, we can note that VARADE (in red) shows the best accuracy without sacrificing too much performance on the inference speed, thus offering the best trade-off on both edge devices. This demonstrates its applicability on constrained devices such as the Jetson Xavier NX, as it offers a significant improvement in accuracy with a minimal loss in the inference speed, still meeting the resource constraints imposed by an edge device.

**Table 2: Comparison between the Anomaly Detection models executed in real-time on the two edge processing units.**

| Board Model      | Anomaly Detection Model | CPU Usage (%) | GPU Usage (%) | RAM Usage (MB) | GPU RAM Usage (MB) | Power Consumption (W) | AUC-ROC | Inference Frequency (Hz) |
|------------------|-------------------------|---------------|---------------|----------------|--------------------|-----------------------|---------|--------------------------|
| Jetson Xavier NX | Idle                    | 36.465        | 52.100        | 5,130.219      | 537.235            | 5.851                 | -       | -                        |
|                  | AR-LSTM                 | 62.311        | 97.700        | 5,669.830      | 872.374            | 11.288                | 0.719   | 5.200                    |
|                  | GBRF                    | 61.499        | 53.000        | 5,518.050      | 528.416            | 6.108                 | 0.655   | 20.575                   |
|                  | AE                      | 53.023        | 79.400        | 5,276.139      | 807.528            | 6.010                 | 0.810   | 2.247                    |
|                  | kNN                     | 92.547        | 55.700        | 5,076.605      | 526.844            | 7.208                 | 0.718   | 1.116                    |
|                  | Isolation Forest        | 51.122        | 64.700        | 4,859.356      | 526.673            | 5.777                 | 0.629   | 4.568                    |
|                  | VARADE                  | 52.420        | 70.600        | 5,488.874      | 1,005.369          | 6.333                 | 0.844   | 14.937                   |
| Jetson AGX Orin  | Idle                    | 4.875         | 0.000         | 3,916.715      | 243.289            | 7.522                 | -       | -                        |
|                  | AR-LSTM                 | 10.744        | 87.200        | 4,741.666      | 761.107            | 11.139                | 0.719   | 8.687                    |
|                  | GBRF                    | 10.475        | 15.900        | 4,279.286      | 245.287            | 9.741                 | 0.655   | 44.128                   |
|                  | AE                      | 10.548        | 51.800        | 4,882.850      | 699.010            | 10.168                | 0.810   | 4.284                    |
|                  | kNN                     | 91.506        | 0.000         | 4,201.195      | 243.289            | 16.887                | 0.718   | 4.754                    |
|                  | Isolation Forest        | 10.648        | 0.000         | 3,990.171      | 243.289            | 9.169                 | 0.629   | 10.732                   |
|                  | VARADE                  | 10.399        | 70.100        | 5,167.490      | 954.701            | 10.220                | 0.844   | 26.461                   |

## 5 CONCLUSIONS AND FUTURE WORKS

In this research we introduced VARADE, a variational based autoregressive system, to address the challenges posed by real-time anomaly detection on the edge. When benchmarked against conventional algorithms, VARADE demonstrated superior performance, while maintaining a significantly higher inference speed compared to other anomaly detection techniques. This positions VARADE as a promising solution, especially in applications that can benefit from the ability to detect complex anomalies. Future works will include experimenting with a larger set of different use cases, to stress the flexibility of our method. Thus, we plan to integrate VARADE within the manufacturing control loop, enabling preventive anomaly detection to activate high-level reconfiguration strategies.

## REFERENCES

- [1] Abdelhadi Azzouni and Guy Pujolle. 2017. A Long Short-Term Memory Recurrent Neural Network Framework for Network Traffic Matrix Prediction. (2017). arXiv:1705.05690 [cs.NI]
- [2] Lorenzo Bacchiani, Giuseppe De Palma, Luca Sciallo, Mario Bravetti, Marco Di Felice, Maurizio Gabbriellini, Gianluigi Zavattaro, and Roberto Della Penna. 2022. Low-Latency Anomaly Detection on the Edge-Cloud Continuum for Industry 4.0 Applications: the SEAWALL Case Study. *IEEE Internet of Things Magazine* 5, 3 (2022), 32–37. <https://doi.org/10.1109/IOTM.001.2200120>
- [3] David M Blei, Alp Kucukelbir, and Jon D McAuliffe. 2017. Variational inference: A review for statisticians. *Journal of the American statistical Association* 112, 518 (2017), 859–877.
- [4] Andrew A Cook, Göksel Mısırlı, and Zhong Fan. 2019. Anomaly detection for IoT time-series data: A survey. *IEEE Internet of Things Journal* 7, 7 (2019), 6481–6494.
- [5] Deepak Ghimire, Dayoung Kil, and Seong-heum Kim. 2022. A survey on efficient convolutional neural networks and hardware acceleration. *Electronics* 11, 6 (2022), 945.
- [6] Markus Goldstein and Seiichi Uchida. 2016. A Comparative Evaluation of Unsupervised Anomaly Detection Algorithms for Multivariate Data. *PLOS ONE* 11, 4 (04 2016), 1–31. <https://doi.org/10.1371/journal.pone.0152173>
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [8] Long Hu, Yiming Miao, Gaoxiang Wu, Mohammad Mehdi Hassan, and Iztok Humar. 2019. iRobot-Factory: An intelligent robot factory based on cognitive manufacturing and edge computing. *Future Generation Computer Systems* 90 (2019), 569–577. <https://doi.org/10.1016/j.future.2018.08.006>
- [9] Huiyue Huang, Lei Yang, Yuanbin Wang, Xun Xu, and Yuqian Lu. 2021. Digital Twin-driven online anomaly detection for an automation system based on edge intelligence. *Journal of Manufacturing Systems* 59 (04 2021), 138–150. <https://doi.org/10.1016/j.jmsy.2021.02.010>
- [10] Dohyung Kim, Hyochang Yang, Minki Chung, Sungzoon Cho, Huijung Kim, Minhee Kim, Kyungwon Kim, and Eunseok Kim. 2018. Squeezed convolutional variational autoencoder for unsupervised anomaly detection in edge device industrial internet of things. In *2018 International conference on information and computer technologies (icict)*. IEEE, 67–71.
- [11] Kacper Kubiak, Grzegorz Dec, and Dorota Stadnicka. 2022. Possible Applications of Edge Computing in the Manufacturing Industry—Systematic Literature Review. *Sensors* 22, 7 (2022). <https://doi.org/10.3390/s22072445>
- [12] Rohit Kumar and Neha Agrawal. 2023. Analysis of multi-dimensional Industrial IoT (IIoT) data in Edge-Fog-Cloud based architectural frameworks: A survey on current state and research challenges. *Journal of Industrial Information Integration* (2023), 100504.
- [13] Gen Li and Jason J Jung. 2022. Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges. *Information Fusion* (2022).
- [14] Zheng Li, Mingxing Duan, Bin Xiao, and Shenghong Yang. 2023. A Novel Anomaly Detection Method for Digital Twin Data Using Deconvolution Operation With Attention Mechanism. *IEEE Transactions on Industrial Informatics* 19, 5 (May 2023), 7278–7286.
- [15] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2012. Isolation-based anomaly detection. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 6, 1 (2012), 1–39.
- [16] Yumeng Liu, Hongan Wang, Xu Zheng, and Ling Tian. 2023. An efficient framework for unsupervised anomaly detection over edge-assisted internet of things. *ACM Transactions on Sensor Networks* (2023).
- [17] Tianyuan Lu, Lei Wang, and Xiaoyong Zhao. 2023. Review of Anomaly Detection Algorithms for Data Streams. *Applied Sciences* 13, 10 (2023), 6353.
- [18] Alessio Mascolini, Sebastiano Gaiardelli, Francesco Ponzio, Nicola Dall'Orta, Enrico Macii, Sara Vinco, Santa Di Cataldo, and Franco Fummi. 2023. Robotic Arm Dataset (RoAD): A Dataset to Support the Design and Test of Machine Learning-Driven Anomaly Detection in a Production Line. In *IECON 2023-49th Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 1–7.
- [19] Garima Nain, KK Pattanaik, and GK Sharma. 2022. Towards edge computing in intelligent manufacturing: Past, present and future. *Journal of Manufacturing Systems* 62 (2022), 588–611.
- [20] Tie Qiu, Jiancheng Chi, Xiaobo Zhou, Zhaolong Ning, Mohammed Atiquzzaman, and Dapeng Oliver Wu. 2020. Edge Computing in Industrial Internet of Things: Architecture, Advances and Challenges. *IEEE Communications Surveys & Tutorials* 22, 4 (2020), 2462–2488. <https://doi.org/10.1109/COMST.2020.3009103>
- [21] Koushik Roy, Abtahi Ishmam, and Kazi Abu Taher. 2021. Demand Forecasting in Smart Grid Using Long Short-Term Memory. In *2021 International Conference on Automation, Control and Mechatronics for Industry 4.0 (ACMI)*. 1–5. <https://doi.org/10.1109/ACMI53878.2021.9528277>
- [22] Haşim Sak, Andrew Senior, and Françoise Beaufays. 2014. Long short-term memory recurrent neural network architectures for large scale acoustic modeling. In *Proc. Interspeech 2014*. 338–342. <https://doi.org/10.21437/Interspeech.2014-80>
- [23] Sebastian Schmidl, Phillip Wenig, and Thorsten Papenbrock. 2022. Anomaly detection in time series: a comprehensive evaluation. *Proceedings of the VLDB Endowment* 15, 9 (2022), 1779–1797.
- [24] Yuting Sun, Tong Chen, Quoc Viet Hung Nguyen, and Hongzhi Yin. 2023. TinyAD: Memory-efficient anomaly detection for time series data in Industrial IoT. *IEEE Transactions on Industrial Informatics* (2023).
- [25] Manu Suvarna, Ken Shaun Yap, Wentao Yang, Jun Li, Yen Ting Ng, and Xiaonan Wang. 2021. Cyber-Physical Production Systems for Data-Driven, Decentralized, and Secure Manufacturing—A Perspective. *Engineering* 7, 9 (2021), 1212–1223.
- [26] Fabrizio De Vita, Giorgio Nocera, Dario Bruneo, and Sajal K. Das. 2023. A Novel Echo State Network Autoencoder for Anomaly Detection in Industrial IoT Systems. *IEEE Transactions on Industrial Informatics* 19, 8 (Aug. 2023), 8985–8994.
- [27] Jingyu Yang and Zuogong Yue. 2023. Learning Hierarchical Spatial-Temporal Graph Representations for Robust Multivariate Industrial Anomaly Detection. *IEEE Transactions on Industrial Informatics* 19, 6 (2023), 7624–7635. <https://doi.org/10.1109/TII.2022.3216006>
- [28] Xiang Yu, Xianfei Yang, Qingji Tan, Chun Shan, and Zhihan Lv. 2022. An edge computing based anomaly detection method in IoT industrial sustainability. *Applied Soft Computing* 128 (2022), 109486.
- [29] Hao Zhou, Ke Yu, Xuan Zhang, Guanlin Wu, and Anis Yazidi. 2022. Contrastive autoencoder for anomaly detection in multivariate time series. *Information Sciences* 610 (2022), 266–280.

# Deep Reinforcement Learning based Online Scheduling Policy for Deep Neural Network Multi-Tenant Multi-Accelerator Systems

Francesco G. Blanco, Enrico Russo, Maurizio Palesi, Davide Patti, Giuseppe Ascia, Vincenzo Catania\*

blanco.francesco@studium.unict.it, enrico.russo@phd.unict.it, {name.surname}@unict.it

University of Catania

Catania, Italy

## ABSTRACT

Currently, there is a growing trend of outsourcing the execution of DNNs to cloud services. For service providers, managing multi-tenancy and ensuring high-quality service delivery, particularly in meeting stringent execution time constraints, assumes paramount importance, all while endeavoring to maintain cost-effectiveness. In this context, the utilization of heterogeneous multi-accelerator systems becomes increasingly relevant. This paper presents RELMAS, a low-overhead deep reinforcement learning algorithm designed for the online scheduling of DNNs in multi-tenant environments, taking into account the dataflow heterogeneity of accelerators and memory bandwidths contentions. By doing so, service providers can employ the most efficient scheduling policy for user requests, optimizing Service-Level-Agreement (SLA) satisfaction rates and enhancing hardware utilization. The application of RELMAS to a heterogeneous multi-accelerator system composed of various instances of Simba and Eyeriss sub-accelerators resulted in up to a 173% improvement in SLA satisfaction rate compared to state-of-the-art scheduling techniques across different workload scenarios, with less than a 1.5% energy overhead.

## 1 INTRODUCTION

Modern real-world applications present increasingly sophisticated challenges that demand equally complex solutions. Among various approaches, Deep Learning (DL), particularly Deep Neural Networks (DNNs), has demonstrated unparalleled capability in addressing these complex problems across diverse applications such as image recognition, object detection, natural language processing, IoT, social networks, medical diagnosis, drug discovery, autonomous driving, and service robotics. As a result, the demand for and reliance on DNNs have significantly grown in various domains.

However, executing DNNs is neither straightforward nor inexpensive due to extensive hardware requirements. With the increasing number of DL models developed daily, incorporating a rising number of parameters, greater memory, and computational resources become necessary [21]. Although computational requirements continue to increase, the same cannot be said for computational power, due to the slowdown of Moore's law and Dennard scaling. A common solution nowadays is to use Domain-Specific Accelerators (DSAs) for executing DNNs.

On the other hand, applications seeking fast, easy, and cost-effective solutions are increasingly delegating DNN execution to cloud services, which benefit from the computational power of large accelerators available in data centers. In the field of Engineering, there is a trend towards simplifying complex tasks, making them accessible to users and developers without extensive technical knowledge. This trend is evident in paradigms like FaaS (Function-as-a-Service, *i.e.*, Serverless Computing) [3], which is now extended to Machine Learning tasks through the INFaaS (Inference-as-a-Service) paradigm [17]. Software developers are delegating such tasks to cloud services, enabling them to focus on core application aspects. As a result, efficient hardware resource management becomes crucial for service providers.

A promising near-term solution is to develop scheduling algorithms that efficiently map different neural networks from different tenants onto the same hardware resource, which typically is a multi-accelerator system (MAS), composed of cores called sub-accelerators (SAs). These algorithms must guarantee user-provided constraints, particularly ensuring that DNN model instances are executed within specified maximum deadlines.

This paper presents RELMAS, an approach to address the challenge of concurrent scheduling of various DNN models on a MAS with heterogeneous SAs. The main objective is to ensure diverse Quality of Service (QoS) levels, particularly in meeting maximum deadline constraints. RELMAS is an online scheduling method that creates a model of the environment, consisting of a MAS with a fixed number of SAs, and a varying number of instances of different DNN models that arrive online and need to be scheduled and completed within specified maximum deadlines. The proposed solution employs Deep Reinforcement Learning, specifically the Deep Deterministic Policy Gradient (DDPG) algorithm. We decided to unconventionally coordinate DDPG algorithm with Long-Short-Term-Memory (LSTM) networks, thus providing a technique to automatically understand deadline constraints among different DNN models instances and provide a schedule that effectively balances the workload through space and time. The algorithm effectively converges, resulting in a substantial reduction in deadline misses and a significant increase in deadline hits.

## 2 RELATED WORK

While the topic of multi-tenancy DNN scheduling in conventional general-purpose cores has been widely studied in both software and hardware since the introduction of chip-multi-processors, addressing multi-tenancy execution in DSAs is a more recent endeavor. Diverse approaches have emerged from academic research that can be classified into two categories [11]: static and dynamic. Static mechanisms involve configuring software or hardware statically (*i.e.*, no decisions are taken at runtime) to adjust the memory access rates of contentious applications. Dynamic mechanisms, on the other hand,

\* F. G. Blanco and E. Russo contributed equally to this research.

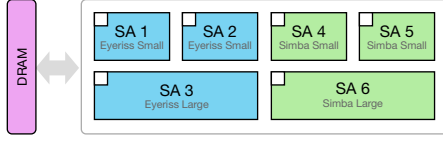
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06

<https://doi.org/10.1145/3649329.3657319>



**Figure 1: Diagram of the reference Multi-Accelerator Heterogeneous Architecture used in evaluations.**

utilize runtime information to adaptively adjust the contentious nature of an application based on the actual amount of memory traffic in the system.

Prior works like LayerWeaver [15], PREMA [5], and AI-MT [2] propose time-multiplexing DNN execution, either statically or dynamically. However, they often suffer from low hardware utilization, especially when layers cannot fully utilize all available resources. To enhance hardware utilization, static partition techniques such as HDA [12] and MAGMA [10] propose spatial partitioning of compute or memory resources. However, their static nature hinders adaptability to different dynamic scenarios. While AI-MT employs multi-accelerators with homogeneous cores (specifically, systolic array architecture), MAGMA also supports multi-accelerators with heterogeneous cores. Additionally, the multi-tenancy support in AI-MT relies on heuristics, while MAGMA uses a research process based on a genetic algorithm. AI-MT considers potential inter-layer dependencies, unlike MAGMA. MAGMA is an offline scheduling strategy, requiring all job characteristics and arrival times to be known beforehand, which does not align with the requirements of on-demand services. Planaria [8] and Veltair [14] propose dynamic allocation of compute resources for co-running workloads. Despite their ability to adaptively allocate compute resources, such as processing elements for Planaria or CPU cores for Veltair, they lack the capability to dynamically manage memory resources. In contrast, MoCA [11] highlights the importance of memory-centric resource management, proposing a dynamic partition of both compute and memory resources throughout execution.

Although homogeneous-core accelerators have been thoroughly explored in the literature, heterogeneous architectures, often incorporating varied dataflows, enable more efficient processing of a diverse range of DNN layers, serving as both flexible and cost-effective solutions [20]. However, in a data center environment that provides on-demand inference services – with unpredictable job arrivals and Service Level Agreements primarily focused on execution within set deadlines – the necessity arises for RELMAS, a sample-efficient, online DNN scheduling algorithm, that mitigates the risk of resource underutilization and QoS violations while maintaining bandwidth constraints.

### 3 PROBLEM FORMULATION

We consider a fixed hardware accelerator architecture, which consists of  $M$  heterogeneous sub-accelerators (SAs), such as the one depicted in Fig.1. In the context of this multi-accelerator system (MAS), individual SAs exhibit distinct specifications encompassing dataflow capabilities, processing unit counts, memory hierarchy configurations, and buffer sizes. All the SAs share the off-chip memory bandwidth. Such a system, which can ideally be deployed in both data center and edge scenarios, operates in a multi-tenant fashion, meaning it can receive and fulfill inference requests from multiple users. The resulting workload within the MAS can be parallelized at

the layer granularity, allowing for concurrent processing of different layers on different SAs.

An user request comprises the task of executing a DNN model inference while adhering to latency constraints specified in a Service Level Agreement (SLA). We refer to each request as a job. Each job consists of several sub-jobs (SJs): one for each layer of the requested DNN model. At runtime, all the sub-jobs to be executed are collected in a virtually infinite ready queue (RQ). SJs are added to the RQ when a new request arrives and are removed when they start their execution on a SA or their deadline (stemming from the SLA) passes.

The objective of this work is to design a scheduling algorithm with the primary goal of maximizing the SLA satisfaction rate, namely the fraction of jobs completed while meeting the specified latency requirements, or conversely minimizing the deadline misses.

We make the reasonable assumption that all potential DNN models that may be requested are known in advance. This is in line with prior works [19]. Layer dependencies and parameters of all these models are known. Thus, it is possible to compile a table containing: the latencies  $c_{i,s}^m$  and the required bandwidths  $b_{i,s}^m$  to execute the  $s$ -th SJ of the  $i$ -th potential request on the  $m$ -th SA. Due to the heterogeneity of the MAS, each layer executed on different SAs exhibits varying latency, energy, and bandwidth requirements. However, in an online context, during periods of low system activity, any unfamiliar model could undergo a “registration phase”, involving compiling latency tables for the new model. Once registered, these models are integrated into the system.

On the other hand, arrival times of each request  $a_j$  are not known in advance. Thus, an online scheduling strategy is needed. Such a scheduler should determine the start time  $s_j$  for each SJ and the SA on which the SJ will be executed. A deadline miss will happen if the finish time  $f_j > s_j + q_j$ , where  $q_j$  is the latency requirement of the request. The finish time cannot be simply calculated summing the computational costs of each layer on the assigned SA because of the limited memory bandwidth shared by all the SAs. When the sum of the bandwidth required by multiple SJs executing in parallel on multiple SAs exceeds the available memory bandwidth, all the SJs experience a slowdown proportional to the respective required bandwidth. This translates to the same amount of stall cycles for all the overlapping sub-jobs. Hence, it is important for the online scheduler to wisely orchestrate the execution of compute-intensive and memory-bound sub-jobs in the RQ to minimize bandwidth contentions.

The described problem can be considered as a generalization of the Flexible Job Shop Scheduling (FJSP) problem and, in Graham’s notation [9], it can be expressed as  $FJSP | prec | U$ , where  $prec$  refers to the presence of precedence constraints among the SJs and  $U$  is the number of tardy jobs, which is the objective to be minimized. It has been proved that the FJSP problem is NP-hard [7].

### 4 RELMAS ONLINE SCHEDULER

In this section we introduce RELMAS, the proposed approach to online scheduling in a multi-accelerator heterogeneous system based on Deep Reinforcement Learning (DRL). For this section, we adopt a top-down approach: we first depict the working of RELMAS in a deployment setting, then we deepen the RELMAS agent learning process. For an introduction to the fundamentals of deep reinforcement learning, please refer to [1].

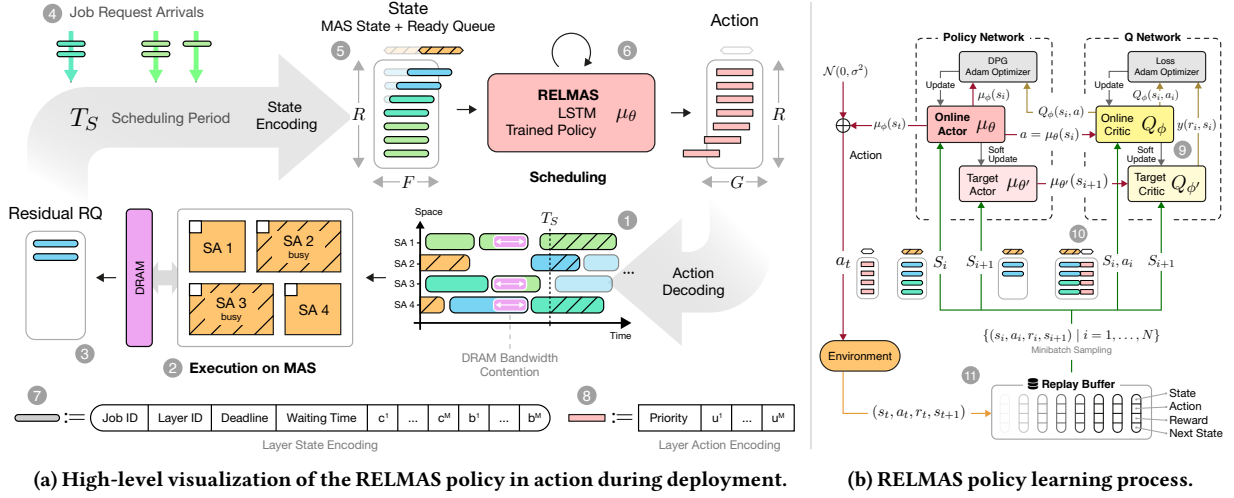


Figure 2: Overview of the proposed online scheduler in production and learning phases.

#### 4.1 Scheduling

In the target scenario, the environment consists in the MAS executing DNN layer inferences and receiving new requests. The latter represent the stochastic component of the environment. The state of the environment at any given time includes the current system load information, *i.e.*, the number of cycles for which each SA will be busy executing a previously assigned SJ, and the ready queue (RQ), *i.e.*, the collection of SJs ready to be executed. The action, which is the decision taken by the RELMAS policy agent for each SJ, can be articulated in a priority, defining temporal scheduling, and a SA choice, defining spatial allocation of the SJ in the MAS. Fig. 2a shows an overview of RELMAS in a deployment setting. It can be noted that the scheduler is triggered periodically with a period  $T_s$ . During a period, the MAS execute the latest elaborated schedule, which is represented graphically in the space-time graph in Fig. 2.1. At the end of the period, SJs remained unexecuted are collected in the so called residual RQ (Fig. 2.3). These SJs are merged with the new SJs related to job requests arrived and queued in the meantime (Fig. 2.4).

This overall RQ (Fig. 2.5) is sorted in ascending order of absolute deadline and submitted for scheduling by the RELMAS policy. We decided to implement the RL policy as a Long-Short-Term-Memory (LSTM) recurrent neural architecture. The rationale is that it helps dealing with a state of variable size (variable number of SJs in RQ) and LSTMs are good in detecting temporal relationships and patterns of SJs sequences. The LSTM cell with a hidden size of  $h$  is succeeded by two fully connected layers: the first projects to  $h/2$  units and applies a Rectified Linear Unit (ReLU) activation function, while the second projects to  $G$  values and applies Hyperbolic Tangent (Tanh) activation function, encoding the decision, *i.e.*, the action, for the corresponding SJ.

As shown in Fig. 2.6, the LSTM sequentially ingests the encoded representation of each SJ in the RQ. Hence, if  $R$  sub-jobs reside in the RQ,  $R$  timesteps of the LSTM policy are executed. The encoded representation (Fig. 2.7) of each SJ is an array of features including all the information that could be useful to derive an optimal scheduling: an identifier of the model requested of which the SJ represent

a layer, an identifier of the layer, the deadline representing the latency constraint<sup>1</sup>, the waiting time informing about the distance in time from the arrival time of the request, the computational times and bandwidth requirements on the SAs. Thus, the length  $F$  of each encoding is equal to  $4+2M$ .

In Fig. 2.5 it is also possible to spot an extra element prepended to the encoded representation of the RQ. As no preemption is assumed, it is important to instruct the policy about the duration for which each SA will be busy, we do this by using a virtual SJ as a primer for the LSTM. All its features are zero except for computational times that instead represent the number of cycles before each SA will be available.

As output of the LSTM policy, after discarding the first timestep (corresponding to the primer) we get a decision encoding (Fig. 2.8) for each SJ in the ready queue. Due to the Tanh activation function, all the output values relies in the  $[-1, 1]$  range that for each SJ are: the temporal priority and one value  $u_m$  for each SA. The index associated with the highest  $u_m$  determines the spatial allocation of the SJ in the MAS. Thus, the length  $G$  of each encoding is equal to  $1+M$ .

#### 4.2 Learning

As shown in Fig. 2b, we employ Deep Deterministic Policy Gradient (DDPG) which has emerged as a prominent algorithm in the field of reinforcement learning, particularly for solving high-dimensional continuous action space problems. For the sake of brevity, we will not describe all the mathematical details of DDPG, please refer to [13] for a detailed explanation. In this section, we instead focus on the choices made to adapt DDPG to the problem addressed by this paper.

As in other RL algorithms, the goal in DDPG is to learn an optimal policy that maximizes a cumulative reward signal over time. To accomplish this, DDPG employs two essential components: the actor and the critic networks. Both are aimed to construct parameterized functions that iteratively converge to the corresponding optimal functions. Due to the exceptional capability of DNNs to approximate non-linear functions, two neural networks are used for these components. The actor network represent the parameterized deterministic

<sup>1</sup>The deadline for each job request is explicitly stated and reiterated within the representation of each SJ. This redundancy enables the policy to differentiate between two identical SJs (same layer, same model), each associated with distinct request latency constraints.

policy function  $\mu_\theta$  and provides a continuous action output given an input state representation. On the other hand, the critic network estimates the Q-function  $Q_\phi$ , which quantifies the expected cumulative reward of following a given policy for a specific state and action pair.

We already discussed the policy as an LSTM-based architecture. Likewise, we propose to adopt the same architecture for the critic network (Fig. 2.9). The state and action pairs are concatenated horizontally as shown in Fig. 2.10 and provided as input to the critic for Q-value estimation. Thus, the input length at each timestep is equal to  $F+G$ , while only one output value  $Q_\phi(s,a)$  per timestep is projected from the last hidden state.

As in the baseline DDPG algorithm, experiences are stored in a *replay buffer* (Fig. 2.11) and sampled from it during the training phase for weight updates. Each experience is a tuple  $(s_t, a_t, r_t, s_{t+1})$ :  $s_t$  is the environment state encoding at time  $t$ ,  $a_t$  is action representation of the decision taken by the policy,  $r_t$  is the reward value obtained as an effect and  $s_{t+1}$  is the state representation after executing the action. As we assume RELMAS to work in a periodic fashion and due to the stochastic request arrivals,  $s_{t+1}$  can also include stochastic request arrivals, therefore breaking a deterministic causality chain from  $s_t$  to  $s_{t+1}$ . Indeed, we found out that learning performance increases when  $s_{t+1}$  only encodes the residual RQ instead of the overall RQ, eliminating the stochastic component from the experiences and allowing the algorithm to effectively infer causality chains.

Regarding the reward function, we defined it as follows:

$$r_t = \sum_{l \in RQ_t} \Delta_{t,l} \left( A_{t,l} + \gamma \frac{(a_l + q_l) - f_l}{q_l} \right)$$

$$\Delta_{t,l} = \begin{cases} 1, & f_l < t + T_s \\ \delta, & \text{otherwise} \end{cases} \quad A_{t,l} = \begin{cases} \alpha, & f_l \leq a_l + q_l \\ -\beta, & \text{otherwise} \end{cases}$$

where  $\delta \in [0,1]$  quantifies the relevance of events beyond the next scheduling period,  $\alpha$  is the deadline hit reward,  $\beta$  is the deadline miss penalty and  $\gamma$  weights the significance of SJs normalized slacks. The introduction of normalized slacks, implemented as a scaled sum term, effectively mitigates resource underutilization. While compliance with SLA requirements is the foremost priority, this strategy additionally incentivizes the reduction of resource usage duration. This alignment serves the dual objective of not only fulfilling service commitments but also boosting operational efficiency.

## 5 EVALUATION

In this section, we present the evaluation of our novel online scheduling technique. To conduct our evaluation, we considered the hardware specifications detailed in Table 1. Our chosen hardware configuration is rooted in a heterogeneous accelerator architecture following the principles described in [12], boasting an equivalent off-chip memory bandwidth and total number of MAC units as in other works focusing on systolic arrays [11]. We are aiming for a multi-chip-module-based architecture in which sub-accelerators take the form of chiplets interconnected by means of a network-on-package (NoP). The on-package links utilize a silicon interposer and incorporate efficient intra-package signaling circuits, as in [18]. The selected architectural configuration comprises six sub-accelerators, which are instances of two widely recognized state-of-the-art DNN accelerators, as visualized in Fig.1. Specifically, half of these sub-accelerators adopt a row-stationary dataflow and an internal organization closely

**Table 1: Parameters of the sub-accelerators classes considered in evaluations.**

| Parameter             | Eyeriss              |                   | Simba             |                |
|-----------------------|----------------------|-------------------|-------------------|----------------|
|                       | Small                | Large             | Small             | Large          |
| Frequency             | 1 GHz                |                   |                   |                |
| DRAM Bandwidth        | 16 GB/s              |                   |                   |                |
| NoP Bandwidth, Energy | 100 GB/s, 1.3 pJ/bit |                   |                   |                |
| Dataflow              | Row Stationary       | Weight Stationary | Weight Stationary | Row Stationary |
| Num. PEs              | 256                  | 512               | 16                | 32             |
| MACs per PE           | 1                    | 1                 | 16                | 16             |
| Global Buffer         | 64 KiB               | 64 KiB            | 32 KiB            | 64 KiB         |
| PE Buffers            | 220 B                | 220 B             | 24 KiB            | 24 KiB         |

**Table 2: Benchmark workloads used in evaluations.**

| Workload | Domain                                          | DNN Models                                  |
|----------|-------------------------------------------------|---------------------------------------------|
| Light    | Image Classification<br>Object Detection<br>NLP | SqueezeNet<br>YOLO-Lite<br>Keyword Spotting |
| Heavy    | Image Classification<br>Object Detection        | AlexNet, InceptionV3, ResNet50<br>YOLO-V2   |
| Mixed    | Light $\cup$ Heavy                              |                                             |

resembling Eyeriss[4], while the remaining half employ a weight-stationary dataflow and an architecture akin to a Simba [18] chiplet. We have chosen to integrate both small and large instances from these two categories of sub-accelerators (see Table 1). This strategic decision enables more effective orchestration of memory-bound and compute-intensive workloads.

In our evaluations we consider seven different state-of-the-art DNN models from image classification, object detection and natural language processing (NLP) domains. These DNN models features different model sizes, computational, memory and bandwidths requirements rendering them highly representative of a wide spectrum of DNN workloads. Similarly to [8] and [11], we organized these models in three workload sets classified by size as summarized in Table 2. To generate multi-tenant scenarios from each of these three workload sets, we consider a time interval during which inter-arrival times of inference requests are drawn from a Pareto distribution, emulating task dispatching in data centers according to insights from [6].

Each request is associated with a SLA requirement defined by the QoS level. Adopting the PREMA [5] approach, the latency requirement is calculated multiplying a coefficient, that we name QoS factor, by the minimum execution latency of the requested job (*i.e.*, without interference of other jobs). Specifically, we identify a QoS factor for the baseline QoS level (QoS-Medium), then we derive other two targets: QoS-Low which indicates relaxed latency constraints corresponding to  $1.2 \times$  QoS-Medium and QoS-High denoting  $0.8 \times$  QoS-Medium, similarly to [8] and [11].

We assess the energy, latency and memory bandwidth requirements figures of each DNN layers on each sub-accelerator using the validated Timeloop/Accelergy [16] simulation infrastructure. Additionally, we develop a multi-accelerator multi-tenant simulation platform that takes into account layer latencies, layer dependencies and runtime memory bandwidth contentions to accurately determine the start and finish times of each sub-job when executing according to a specific scheduling algorithm. This methodology allows us to effectively evaluate the target metrics. Across all the evaluations, the

following reward coefficients were used during RELMAS learning:  $\alpha = 0.10$ ,  $\beta = 0.11$ ,  $\gamma = 0.05$  and  $\delta = 0.01$ .

## 5.1 SLA Satisfaction Rate

In Fig. 3 we show the results of the evaluation of the three workload sets in Table 2 with three different QoS targets. We adopt the SLA Satisfaction Rate as our primary comparative metric. To evaluate the efficacy of the proposed approach, we conduct comparative analyses against robust baselines, which draw inspiration from and are adapted from previous works in the field. The following baseline algorithms are considered: (1) FCFS-H, which is the First-Come-First-Served scheduling algorithm for priority assignment to each sub-job in the ready queue, paired with an heuristic for the selection of the sub-accelerator. The heuristic consists in choosing the SA which minimizes the finish time of the sub-job. (2) PREMA-H, which is a scheduling algorithm based on [5], combining a token mechanism based on waiting time and Shortest-Job-First priority assignment. As the original PREMA work targets a monolithic architecture, for a fair comparison we combine it with the aforementioned heuristic. (3) Herald [12], which is a layer scheduling algorithm designed for heterogeneous architectures, focusing on load balancing of the SAs. (4) MAGMA [10], consisting in a genetic algorithm approach to scheduling optimization. Our implementation adopts the same custom genetic operators of the original work. For a fair comparison, we used the same evaluation platform as RELMAS and adapted its fitness function to take into account the SLA satisfaction rate.

All the evaluated algorithms are supposed to be triggered for each scheduling period at runtime. For each tested scenario we trained RELMAS considering a hidden size of 256 for both the actor and the critic. As shown in Fig. 3, the trained policy is able to match or outperform the other methods across all the scenarios in terms of SLA satisfaction rate.

With respect to Herald, RELMAS achieves a geometric mean improvement of 59.4%, while with respect to PREMA-H it achieves a geometric mean improvement of 109%. The greatest improvements with respect to these two baselines are achieved for the Mixed workload. We believe, this effect should be attributed to high variability of model sizes in the Mixed workload, that cause the scheduling process to be more complex. In particular, the Shortest-Job-First strategy of PREMA, allocates more priority to the Light models in the Mixed workload starving the Heavy models.

RELMAS achieves the same SLA Satisfaction Rate of MAGMA in the Light and Heavy workloads. However, it is worth noting that MAGMA is executed with same parameters of the original paper, *i.e.*, 100 generations of 100 individuals, for each scheduling period to optimally scheduling a ready queue. For the evaluation of individual fitness, we employ the identical simulation platform, previously delineated and utilized for RELMAS training. These choices resulted in MAGMA scheduling runtimes of several minutes on a standard workstation for each simulated scheduling period, which is reasonable as MAGMA, unlike RELMAS, was born to be an offline scheduling policy, not an online one. While this places our MAGMA implementation in the realm of limited practicality within real-world contexts, it serves as a strong benchmark for evaluating the proposed approach. Furthermore, RELMAS is capable to outperform MAGMA in a Mixed workload workload scenario with up to 22.5%. SLA Satisfaction Rate improvement. We will analyze the overhead of the proposed solution in Sec. 5.3.

## 5.2 Bandwidth Sensitivity

RELMAS is designed to be aware of the bandwidth required by each DNN layer to be executed without stalls. To this end, this information is embedded as a feature in encoded LSTM policy input. This makes the proposed technique more robust in memory-constrained contexts. We test a Light workload scenario and in Fig. 4 we show the SLA Satisfaction Rate of different scheduling strategies normalized to their respective best result achieved. It can be noted that, independently from their absolute performance, some scheduling approaches tend to experience an abrupt degradation when decreasing the memory bandwidth, with respect to their performance in a bottleneck-free situation. On the other hand, RELMAS degradation is slower, especially in the low-bandwidth region, indicating an higher capability of the proposed approach to better orchestrate layers on the heterogeneous hardware optimizing memory bandwidth utilization. We found that the same conclusions remain valid for the other considered workloads; therefore, we are reporting the results for the Light workload only.

## 5.3 Overhead Assessment

We now analyze the overhead of the proposed solution for the online scheduling in a multi-accelerator system. As already described, RELMAS consists in a DNN model composed of an LSTM cell followed by two projection fully connected layers. The policy works recurrently ingesting one input state encoding at each timestep, one for each layer in the ready queue. To gain insight into the computational complexity, it is informative to note that for a policy with an hidden size of 256, the number of Multiply-Accumulate (MAC) operations per layer amounts to 316 288, representing about a mere 0.04% and 0.007% of the MAC operations required by the AlexNet and ResNet50 models respectively. We opted to implement the scheduling policy within one of the accelerators designated for the policy's execution. This was achieved by communicating the accelerator's busy status to the scheduler, which was done by adding the overhead time to the first element in the State encoding, as shown in Fig. 2.5. Our experiments revealed that this implementation incurred no overhead impact on the scheduling process itself. We conducted an accurate evaluation of the energy overhead assuming that the scheduler is executed on the MAS itself. In particular, we assumed that one of the Simba Small SAs (Fig. 1) is responsible for running each timestep of the RELMAS policy. We evaluated the energy required for this setup using Timeloop [16]. We then assessed the overall energy overhead to the workload execution considering different hidden sizes for the LSTM cell and the Mixed workload. The results are shown in Fig. 5. We found that, for the considered workload, there is no significant SLA Satisfaction Rate improvement for an hidden size bigger than 128. In any case, the energy overhead does not exceed 1.3%. We also noticed that decreasing the scheduling period can increase the performance of the scheduler, because it enables shorter reaction times. This comes to the cost of a slight increase in energy overhead due to the increasing average length of residual ready queues. In other words, a layer will be scheduled multiple times on average before being actually executed, because it is often forwarded to the next scheduling period.

## 6 CONCLUSION

This paper presents RELMAS, a low-overhead deep reinforcement learning based online scheduling policy for DNN multi-tenant heterogeneous multi-accelerator systems. We believe that reinforcement learning could be a viable and low-overhead solution for online

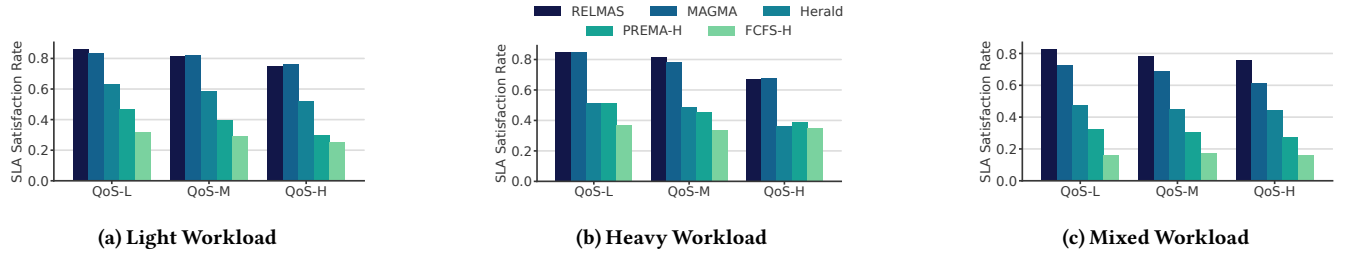


Figure 3: SLA Satisfaction Rate comparison against other baselines for different workload sets.

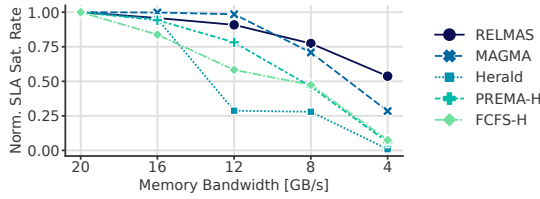


Figure 4: Impact of memory bandwidth reduction on SLA Satisfaction Rate for different scheduling strategies.

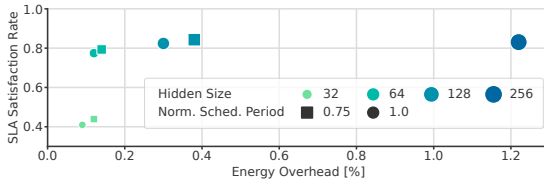


Figure 5: Energy overhead of the proposed scheduling algorithm varying the hidden size of the LSTM policy and the scheduling period.

scheduling and dispatching in multi-accelerator multi-tenant heterogeneous systems. For the proposed methodology, we included computation times and memory bandwidths requirements as features for the RL policy. We think many more could be explored to enable other functionalities. For instance, tenant-wise SLA satisfaction and fairness awareness could be enabled by exploring new state encoding and online tenant-wise deadline misses closed-loop control with dynamic reward shaping. Furthermore, we unconventionally considered an LSTM architecture for the policy but other trending architectures could be explored and their overhead assessed such as transformer-based models.

## ACKNOWLEDGMENTS

This work has been (partially) supported by MUR project ARS01\_00592 reCITY. The work of M. Palesi, who has contributed to the development of Secs. 3 and 4, has been supported by the Spoke 1 "FutureHPC & BigData" of the Italian Research Center on High-Performance Computing, Big Data and Quantum Computing (ICSC). The work of V. Catania, who has contributed to Sec. 5, has been supported by PNRR MUR project PE0000013-FAIR.

## REFERENCES

- [1] K. Arulkumar, M. P. Deisenroth, M. Brundage, and A. A. Bharath. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, 34(6):26–38, 2017.
- [2] E. Baek, D. Kwon, and J. Kim. A multi-neural network acceleration architecture. In *ACM/IEEE International Symposium on Computer Architecture*, 2020.
- [3] I. Baldini et al. *Serverless Computing: Current Trends and Open Problems*, pages 1–20. Springer Singapore, 2017.
- [4] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 9(2):292–308, 2019.
- [5] Y. Choi and M. Rhu. Prema: A predictive multi-task scheduling algorithm for preemptible neural processing units. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 220–233. IEEE, 2020.
- [6] G. Da Costa, L. Grange, and I. De Courchelle. Modeling, classifying and generating large-scale google-like workload. *Sustainable Computing: Informatics and Systems*, 19:305–314, 2018.
- [7] S. Dauzère-Pérès, J. Ding, L. Shen, and K. Tamssaouet. The flexible job shop scheduling problem: A review. *European Journal of Operational Research*, 2023.
- [8] S. Ghodrati et al. Planaria: Dynamic architecture fission for spatial multi-tenant acceleration of deep neural networks. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 681–697. IEEE, 2020.
- [9] R. L. Graham, E. L. Lawler, J. K. Lenstra, and A. R. Kan. Optimization and approximation in deterministic sequencing and scheduling: a survey. In *Annals of discrete mathematics*, volume 5, pages 287–326. Elsevier, 1979.
- [10] S. Kao and T. Krishna. Magma: An optimization framework for mapping multiple dnn on multiple accelerator cores. In *International Symposium on High-Performance Computer Architecture*, pages 814–830, apr 2022.
- [11] S. Kim, H. Genc, V. V. Nikiforov, K. Asanović, B. Nikolić, and Y. S. Shao. Moca: Memory-centric, adaptive execution for multi-tenant deep neural networks. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.
- [12] H. Kwon, L. Lai, M. Pellauer, T. Krishna, Y. Chen, and V. Chandra. Heterogeneous dataflow accelerators for multi-dnn workloads. In *International Symposium on High-Performance Computer Architecture*, pages 71–83, mar 2021.
- [13] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv:1509.02971*, 2015.
- [14] Z. Liu, J. Leng, Z. Zhang, Q. Chen, C. Li, and M. Guo. Veltair: Towards high-performance multi-tenant deep learning services via adaptive compilation and scheduling. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 388–401, 2022.
- [15] Y. H. Oh, S. Kim, Y. Jin, S. Son, J. Bae, J. Lee, Y. Park, D. U. Kim, T. J. Ham, and J. W. Lee. Layerweaver: Maximizing resource utilization of neural processing units via layer-wise scheduling. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 584–597, 2021.
- [16] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer. Timeloop: A systematic approach to dnn accelerator evaluation. In *International symposium on performance analysis of systems and software*, pages 304–315. IEEE, 2019.
- [17] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis. INFaaS: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 397–411. USENIX Association, July 2021.
- [18] Y. S. Shao et al. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 14–27, New York, NY, USA, 2019. Association for Computing Machinery.
- [19] J. Soifer, J. Li, M. Li, J. Zhu, Y. Li, Y. He, E. Zheng, A. Oltean, M. Mosyak, C. Barnes, et al. Deep learning inference service at microsoft. In *Conference on Operational Machine Learning*, pages 15–17, 2019.
- [20] A. Symons, L. Mei, S. Colleman, P. Houshmand, S. Karl, and M. Verhelst. Towards heterogeneous multi-core accelerators exploiting fine-grained scheduling of layer-fused deep neural networks. *arXiv:2212.10612*, 2022.
- [21] X. Xu, Y. Ding, S. Hu, M. Niemier, J. Cong, Y. Hu, and Y. Shi. Scaling for edge inference of deep neural networks. *Nature Electronics*, 1:216–222, 2018.